ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC – KỸ THUẬT MÁY TÍNH



Vi xử lý - Vi điều khiển

Báo cáo bài thực hành 3

Nút nhấn và máy trạng thái

GVHD : Phan Văn Sỹ

SV thực hiện : Trần Minh Quang – 2412855

TP. HỒ CHÍ MINH, THÁNG 11/ 2026

Mục lục

1 Đặc tả hệ thống **1**

2 Bài tập 1 **1**

3 Bài tập 2 **3**

4 Bài tập 3 **5**

5 Bài tập 4 **6**

6 Bài tập 5 **8**

7 Bài tập 6 **10**

8 Bài tập 7 **13**

9 Bài tập 8 **15**

10 Bài tập 9 **16**

11 Bài tập 10 **17**

12 Mã nguồn cho bài thực hành 3 **20**

1 Đặc tả hệ thống

Bài thực hành này xây dựng ứng dụng đèn giao thông cho một ngã tư trên vi điều khiển STM32F103C6, mô phỏng bằng Proteus. Hệ thống gồm:

- **12 đèn đơn**: 4 đỏ, 4 vàng, 4 xanh, chia cho 2 tuyến đường. Hai hướng đối diện của cùng một tuyến luôn sáng cùng màu.
- **4 LED 7 đoạn**: mỗi tuyến 2 LED, hiển thị số giây còn lại của đèn đang sáng.
- **3 nút nhấn**: BTN1 chọn chế độ, BTN2 tăng giá trị thời gian, BTN3 lưu giá trị.

Hệ thống có 4 chế độ, BTN1 chuyển lần lượt Mode 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 1:

- **Mode 1 – Bình thường**: đèn giao thông hoạt động tự động.
- **Mode 2 – Chỉnh thời gian đèn đỏ**: các đèn đỏ nhấp nháy 2Hz; hai LED 7 đoạn hiển thị giá trị đang chỉnh, hai LED còn lại hiển thị số mode. BTN2 tăng giá trị trong khoảng 1–99, BTN3 lưu giá trị.
- **Mode 3, Mode 4**: tương tự Mode 2, lần lượt cho đèn vàng và đèn xanh.

Toàn bộ chương trình dùng một ngắt timer duy nhất là TIM2 với chu kỳ 10ms (cấu hình ở bài tập 3). Hàm ngắt chỉ cập nhật các software timer và đọc nút nhấn; mọi xử lý logic được đặt trong vòng lặp `while(1)`.

Báo cáo này trình bày project `Traffic-Light_FSM`, trong đó mã nguồn được tách thành các module theo vai trò:

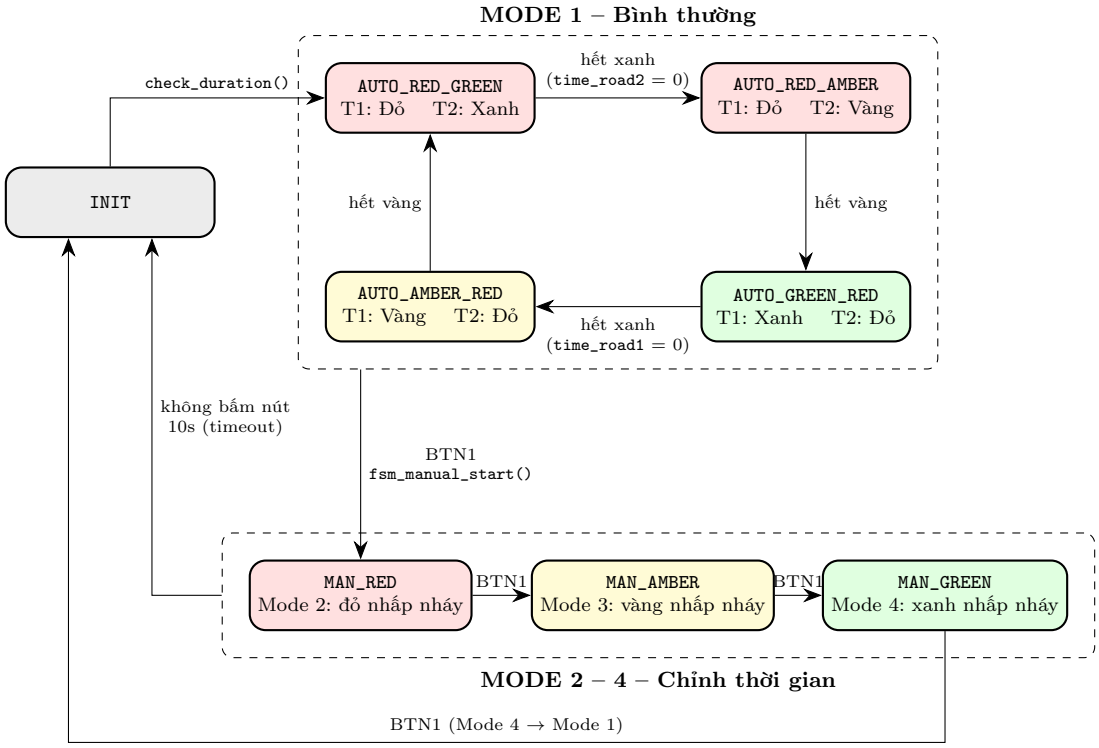
Lớp	File	Nhiệm vụ
Dữ liệu dùng chung	`global.h/.c`	Trạng thái hệ thống, thời gian mỗi màu đèn
Thời gian	`software_timer.h/.c`	Các software timer, đơn vị mili giây
Đầu vào	`button.h/.c`	Đọc nút, chống dội, phát hiện nhấn giữ
Đầu ra	`traffic_light.h/.c`	Điều khiển 12 đèn đơn
Đầu ra	`led7seg.h/.c`	Quét 4 LED 7 đoạn
Xử lý	`fsm_automatic.h/.c`	Máy trạng thái Mode 1
Xử lý	`fsm_manual.h/.c`	Máy trạng thái Mode 2–4

Các file FSM không gọi trực tiếp hàm HAL mà chỉ dùng hàm của các module đầu vào/đầu ra. Nhờ vậy, khi đổi chân hoặc mức logic của đèn, chỉ cần sửa file driver tương ứng.

2 Bài tập 1

Vẽ máy trạng thái (FSM) mô tả giải pháp cho bài toán đèn giao thông ở trên.

Report 1: Sơ đồ máy trạng thái của hệ thống:



Hình 1: Máy trạng thái của hệ thống đèn giao thông

Report 2: Giải thích thiết kế.

Toàn hệ thống dùng một biến trạng thái `status` chung, kiểu `Traffic_State` (khai báo trong `global.h`). Máy trạng thái được chia thành hai phần, mỗi phần nằm trong một file riêng:

- **Mode 1** (`fsm_automatic.c`): gồm trạng thái `INIT` và 4 pha `AUTO_*`. Tên mỗi pha ghép màu của tuyến 1 và tuyến 2, ví dụ `AUTO_RED_GREEN` là tuyến 1 đỏ, tuyến 2 xanh. Thiết kế trạng thái *kết hợp* cả hai tuyến như vậy bảo đảm hai tuyến luôn đồng bộ: không có trạng thái nào mà cả hai tuyến cùng được đi.
- **Mode 2–4** (`fsm_manual.c`): gồm 3 trạng thái `MAN_RED`, `MAN_AMBER`, `MAN_GREEN`, mỗi trạng thái chỉnh thời gian của một màu đèn.

Để hai tuyến khớp pha, thời gian các màu phải thỏa ràng buộc:

$$T_{\text{đỏ}} = T_{\text{xanh}} + T_{\text{vàng}}$$

Trong lúc tuyến 1 đỏ, tuyến 2 lần lượt xanh rồi vàng; hai tuyến chỉ đổi vai cùng lúc khi ràng buộc trên được thỏa. Với giá trị mặc định đỏ 5s, vàng 2s, xanh 3s, một chu kỳ đầy đủ kéo dài $2 \times 5 = 10$s.

Bảng chuyển trạng thái:

Trạng thái	Sự kiện	Trạng thái kế	Hành động
`INIT`	(ngay lập tức)	`AUTO_RED_GREEN`	Kiểm tra ràng buộc thời gian, bật nhịp 1s
`AUTO_RED_GREEN`	Tuyến 2 hết xanh	`AUTO_RED_AMBER`	Tuyến 2 sang vàng, nạp thời gian vàng
`AUTO_RED_AMBER`	Tuyến 2 hết vàng	`AUTO_GREEN_RED`	Đổi vai: tuyến 1 xanh, tuyến 2 đỏ
`AUTO_GREEN_RED`	Tuyến 1 hết xanh	`AUTO_AMBER_RED`	Tuyến 1 sang vàng
`AUTO_AMBER_RED`	Tuyến 1 hết vàng	`AUTO_RED_GREEN`	Đổi vai: tuyến 1 đỏ, tuyến 2 xanh
`AUTO_*`	BTN1	`MAN_RED`	Tắt nhịp 1s, sao lưu thời gian cũ
`MAN_RED`, `MAN_AMBER`	BTN1	Mode kế tiếp	Bỏ giá trị chưa lưu
`MAN_GREEN`	BTN1	`INIT`	Kiểm tra ràng buộc, về Mode 1
`MAN_*`	10s không bấm nút	`INIT`	Kiểm tra ràng buộc, về Mode 1
`MAN_*`	BTN2 (nhấn/giữ)	(giữ nguyên)	Tăng giá trị đang chỉnh (99 $\rightarrow$ 1)
`MAN_*`	BTN3	(giữ nguyên)	Lưu giá trị vào `duration[]`

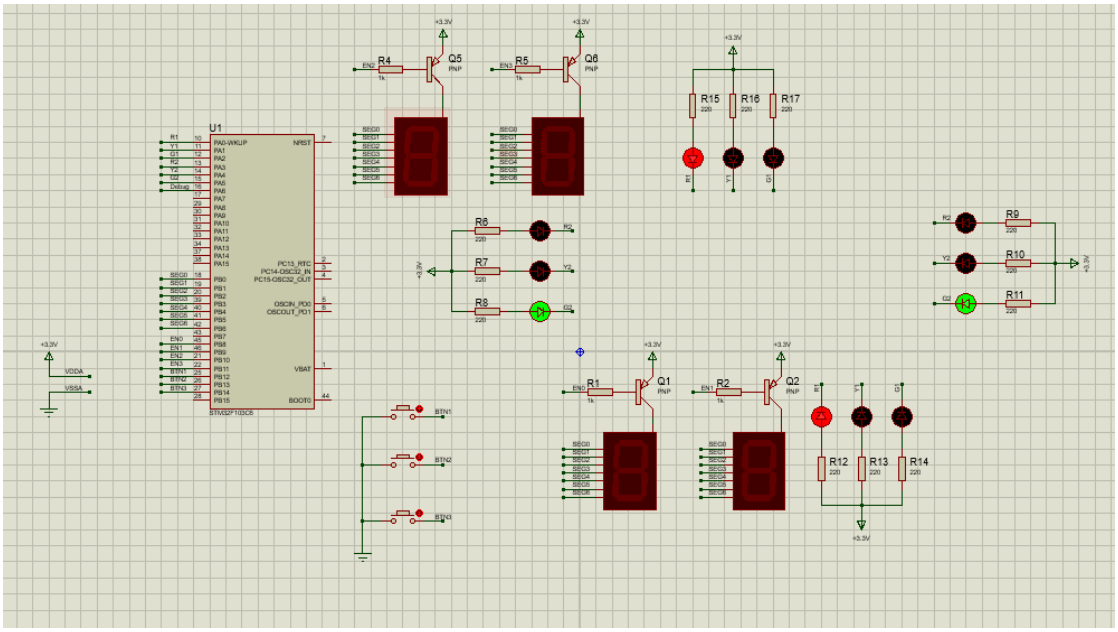
Ngoài đặc tả, hệ thống bổ sung một số chức năng để hoạt động an toàn hơn:

- **Timeout 10s**: ở Mode 2–4, nếu người dùng không bấm nút nào trong 10s, hệ thống tự quay về Mode 1. Nhờ vậy ngã tư không bị kẹt ở chế độ chỉnh khi người vận hành bỏ đi.
- **Kiểm tra ràng buộc khi thoát**: khi rời Mode 2–4, nếu bộ thời gian mới vi phạm $T_{\text{đỏ}} = T_{\text{xanh}} + T_{\text{vàng}}$ thì toàn bộ thay đổi bị hủy và hệ thống dùng lại bộ thời gian trước khi vào chỉnh.
- **Nhấn giữ**: giữ BTN2 thì giá trị tự tăng mỗi giây, không cần bấm liên tục.

3 Bài tập 2

Vẽ sơ đồ mô phỏng Proteus cho bài toán.

Report 1: Sơ đồ mô phỏng trên Proteus:



Hình 2: Sơ đồ nguyên lý hệ thống đèn giao thông

Link github cho file mô phỏng: https://github.com/qanetta/Microprocessor-Lab/blob/main/lab3/proteus_files/Traffic-Light_FSM.pdsprj

Report 2: Mô tả sơ đồ.

- **12 đèn đơn** được chia thành 4 cụm (mỗi hướng một cụm đỏ – vàng – xanh). Mỗi đèn nối theo kiểu *kéo xuống*: `+3.3V` → điện trở 220Ω → LED → chân vi điều khiển, nên đèn sáng khi chân xuất mức 0. Hai cụm của hai hướng đối diện dùng chung 3 chân (R1/Y1/G1 hoặc R2/Y2/G2), vì hai hướng này luôn cùng màu. Nhờ vậy 12 đèn chỉ cần 6 chân.
- **4 LED 7 đoạn** loại Anode chung. Các chân đoạn `SEG0`–`SEG6` nối chung vào PB0–PB6. Anode của mỗi LED được cấp nguồn qua một transistor PNP (Q1, Q2, Q5, Q6), điều khiển bởi EN0–EN3. Cặp LED của Q1–Q2 (EN0, EN1) đặt cạnh cụm đèn tuyến 1, cặp Q5–Q6 (EN2, EN3) đặt cạnh cụm đèn tuyến 2.
- **3 nút nhấn** BTN1–BTN3 nối một đầu xuống GND. Chân vi điều khiển được cấu hình pull-up nội, nên khi thả nút đọc được mức 1, khi nhấn đọc được mức 0.

Bảng nối chân:

Chân	Tên (User Label)	Chế độ	Chức năng
PA0, PA1, PA2	R1, Y1, G1	Output push-pull	Đèn đỏ, vàng, xanh tuyến 1 (tích cực mức 0)
PA3, PA4, PA5	R2, Y2, G2	Output push-pull	Đèn đỏ, vàng, xanh tuyến 2 (tích cực mức 0)
PA6	LED_DEGUB	Output push-pull	LED dự phòng để debug
PB0–PB6	seg_a–seg_g	Output push-pull	7 đoạn a–g (tích cực mức 0)
PB8–PB11	EN0–EN3	Output push-pull	Chọn LED 7 đoạn qua PNP (tích cực mức 0)
PB12–PB14	BTN1–BTN3	Input pull-up	Nút MODE, INC, SET

4 Bài tập 3

Tạo project STM32CubeIDE có cấu hình chân tương ứng với sơ đồ Proteus ở bài tập 2, với ngắt timer khoảng 10ms.

Report 1: Project được tạo cho chip STM32F103C6, dùng nguồn xung nội HSI 8MHz. Các chân được cấu hình trong file `.ioc` theo bảng nối chân ở bài tập 2. CubeMX sinh ra hàm `MX_GPIO_Init` như sau:

```
/*Configure GPIO pins : R1_Pin Y1_Pin G1_Pin R2_Pin
                        Y2_Pin G2_Pin LED_DEGUB_Pin */
GPIO_InitStruct.Pin = R1_Pin|Y1_Pin|G1_Pin|R2_Pin
                      |Y2_Pin|G2_Pin|LED_DEGUB_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

/*Configure GPIO pins : seg_a_Pin seg_b_Pin seg_c_Pin EN2_Pin
                        EN3_Pin seg_d_Pin seg_e_Pin seg_f_Pin
                        seg_g_Pin EN0_Pin EN1_Pin */
GPIO_InitStruct.Pin = seg_a_Pin|seg_b_Pin|seg_c_Pin|EN2_Pin
                      |EN3_Pin|seg_d_Pin|seg_e_Pin|seg_f_Pin
                      |seg_g_Pin|EN0_Pin|EN1_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

/*Configure GPIO pins : BTN1_Pin BTN2_Pin BTN3_Pin */
GPIO_InitStruct.Pin = BTN1_Pin|BTN2_Pin|BTN3_Pin;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

Cấu hình GPIO trong hàm **MX_GPIO_Init()**

Report 2: Timer 2 được bật và ngắt TIM2 được kích hoạt trong mục NVIC. Với Prescaler = 7999 và Counter Period = 9, chu kỳ ngắt là:

$$f_{TIM} = \frac{8\,\text{MHz}}{7999+1} = 1000\,\text{Hz}, \qquad f_{\text{ngắt}} = \frac{1000\,\text{Hz}}{9+1} = 100\,\text{Hz} \;\Rightarrow\; T_{\text{ngắt}} = 10\,\text{ms}$$

Hàm ngắt chỉ làm hai việc ngắn: cập nhật các software timer và đọc trạng thái nút nhấn. Thời gian thực thi trong ngắt nhờ vậy rất ngắn và không phụ thuộc vào chế độ đang chạy.

```c
/* USER CODE BEGIN 4 */
// ISR chi lam viec ngan: dem software timer va doc nut. Logic nam het o while(1)
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if (htim->Instance == TIM2){
        timerRun();
        getKeyInput();
    }
}
/* USER CODE END 4 */
```

Hàm ngắt timer

Timer được khởi động trong vùng `USER CODE BEGIN 2`, sau khi tắt toàn bộ đèn:

```c
  /* USER CODE BEGIN 2 */
  traffic_all_off();
  HAL_TIM_Base_Start_IT(&htim2);        // TIM2: 7999/9 -> ngat moi 10ms (TIMER_CYCLE)
  setTimer(TIMER_SCAN, SCAN_PERIOD);
  status = INIT;
  /* USER CODE END 2 */
```

Khởi tạo trong hàm main

5 Bài tập 4

Sửa đổi cách dùng timer sao cho khi thay đổi chu kỳ ngắt, chỉ cần sửa ít nhất và không ảnh hưởng đến hoạt động của hệ thống. Ví dụ: LED nhấp nháy 2Hz với chu kỳ ngắt 10ms vẫn phải nhấp nháy 2Hz khi chu kỳ ngắt đổi thành 1ms hoặc 100ms.

Report 1: Giải pháp là tách software timer thành module riêng và cho các hàm `setTimer` nhận tham số theo **mili giây** thay vì số lần ngắt. Chu kỳ ngắt chỉ được khai báo đúng một lần bằng hằng `TIMER_CYCLE`. Khi đổi chu kỳ ngắt, ta chỉ cần sửa Prescaler/Counter Period trong file `.ioc` và hằng `TIMER_CYCLE`; mọi lời gọi như `setTimer(TIMER_BLINK, 250)` giữ nguyên.

Các software timer được quản lý bằng mảng, đánh chỉ số bằng enum `Timer_ID`. Phần tử cuối `NUM_OF_TIMERS` tự động bằng số lượng timer, nên khi thêm timer mới chỉ cần thêm một tên vào enum.

```c
#define TIMER_CYCLE 10  // chu ky ngat TIM2 (ms) - Ex4: doi chu ky ngat chi sua o day

// Ten cac software timer, phan tu cuoi tu dong la so luong timer
typedef enum{
    TIMER_SECOND = 0,   // nhip 1s: dem nguoc thoi gian den o Mode 1
    TIMER_SCAN,         // quet 4 LED 7 doan
    TIMER_BLINK,        // nhay den 2Hz o Mode 2-4
    TIMER_TIMEOUT,      // 10s khong bam nut o Mode 2-4 -> ve Mode 1
    NUM_OF_TIMERS
} Timer_ID;

void setTimer(int index, int duration); // duration tinh bang ms
int isTimerExpired(int index);              // tra ve 1 neu het thoi gian
void timerRun(void);                        // goi trong ISR moi TIMER_CYCLE ms
```

software_timer.h

```c
#include "software_timer.h"

static volatile int timer_flag[NUM_OF_TIMERS];
static volatile int timer_counter[NUM_OF_TIMERS];

void setTimer(int index, int duration){
    if (index < 0 || index >= NUM_OF_TIMERS) return;
    // lam tron len: luon chay it nhat 1 chu ky ngat
    timer_counter[index] = (duration + TIMER_CYCLE - 1) / TIMER_CYCLE;
    timer_flag[index] = 0;   // xoa co SAU khi nap counter
}

int isTimerExpired(int index){
    if (index < 0 || index >= NUM_OF_TIMERS) return 0;
    return timer_flag[index];
}

void timerRun(void){
    for (int i = 0; i < NUM_OF_TIMERS; i++){
        if (timer_counter[i] > 0){
            timer_counter[i]--;
            if (timer_counter[i] == 0){
                timer_flag[i] = 1;
            }
        }
    }
}
```

software_timer.c

Hai chi tiết quan trọng trong `setTimer`:

- **Làm tròn lên** thay vì chia nguyên. Ở bài thực hành 2, `setTimer0(1)` cho kết quả `1 / 10 = 0` và timer không bao giờ hết hạn. Với công thức $\lceil$`duration/TIMER_CYCLE`$\rceil$, mọi giá trị dương đều chạy ít nhất một chu kỳ ngắt.
- **Xóa cờ sau khi nạp bộ đếm**. Nếu xóa cờ trước, ngắt có thể xảy ra đúng giữa hai lệnh, thấy bộ đếm cũ bằng 0 và không làm gì, dẫn đến timer bị kẹt.

Report 2: Kiểm chứng với LED nhấp nháy 2Hz (`BLINK_MS = 250`) khi đổi chu kỳ ngắt. Với 1ms, phải giảm Prescaler vì Counter Period không được bằng 0 (timer sẽ dừng đếm):

$T_{\text{ngắt}}$	**Prescaler**	**Period**	`TIMER_CYCLE`	**Bộ đếm cho 250ms**	**Thời gian thực**
1ms	799	9	1	250	250ms
10ms	7999	9	10	25	250ms
100ms	7999	99	100	$\lceil 2.5 \rceil = 3$	300ms

Với 1ms và 10ms, đèn vẫn nhấp nháy đúng 2Hz. Với 100ms, thời gian bị làm tròn lên 300ms (khoảng 1.67Hz) vì software timer không thể đếm thời gian mịn hơn một chu kỳ ngắt. Như vậy

cách làm này giữ đúng thời gian *trong giới hạn độ phân giải* `TIMER_CYCLE`; muốn đúng tuyệt đối, các khoảng thời gian trong chương trình phải là bội số của chu kỳ ngắt.

Chu kỳ ngắt cũng ảnh hưởng đến chống dội nút (bài tập 5), vì nút được lấy mẫu mỗi chu kỳ ngắt: 3 mẫu liên tiếp trải dài $2 \times$ `TIMER_CYCLE`. Với 1ms, cửa sổ chỉ còn 2ms, ngắn hơn thời gian dội thường gặp của nút cơ (5–20ms). Với 100ms, cửa sổ là 200ms, nút phản hồi chậm và có thể bỏ sót lần nhấn nhanh. Vì vậy 10ms là lựa chọn hợp lý cho bài này.

6 Bài tập 5

Theo ví dụ đọc nút và chống dội ở phần lý thuyết: thêm các file mới cho việc đọc đầu vào và hiển thị đầu ra, thêm code chống dội nút nhấn, và thêm code tăng mode khi nhấn nút thứ nhất.

Report 1: Các file mới được thêm vào project:

- Đọc đầu vào: `button.h/.c` (tương ứng `input_reading` trong ví dụ).
- Hiển thị đầu ra: `traffic_light.h/.c` cho 12 đèn đơn và `led7seg.h/.c` cho LED 7 đoạn (trình bày ở bài tập 6).
- Xử lý: `fsm_automatic.h/.c` và `fsm_manual.h/.c` (tương ứng `input_processing` trong ví dụ).

Các nút được đặt tên bằng enum, phần tử cuối là số lượng nút. Mức logic khi nhấn và khi thả được khai báo bằng hằng, nên nếu đổi mạch sang nút tích cực mức cao thì chỉ cần sửa hai dòng này:

```c
#define NORMAL_STATE    GPIO_PIN_SET    // pull-up: tha nut doc duoc 1
#define PRESSED_STATE   GPIO_PIN_RESET  // nhan nut noi xuong GND: doc duoc 0

// Ten cac nut, phan tu cuoi tu dong la so luong nut
typedef enum{
    BTN_MODE = 0,   // BTN1: chuyen mode 1 -> 2 -> 3 -> 4 -> 1
    BTN_INC,        // BTN2: tang gia tri thoi gian
    BTN_SET,        // BTN3: luu gia tri
    NUM_OF_BUTTONS
} Button_ID;

void getKeyInput(void);             // goi trong ISR moi TIMER_CYCLE ms: doc chan + chong doi
int isButtonPressed(int index);     // tra ve 1 MOT LAN cho moi lan nhan, roi tu xoa co
int isButtonLongPressed(int index); // tra ve 1 khi dang giu nut lau hon 1s
```

button.h

Report 2: Code chống dội. Hàm `getKeyInput()` được gọi trong ngắt timer mỗi 10ms và dùng 4 thanh ghi cho mỗi nút:

- `keyReg0`, `keyReg1`, `keyReg2`: 3 mẫu đọc gần nhất, hoạt động như một thanh ghi dịch (`keyReg2` là mẫu mới nhất). Trạng thái chỉ được chấp nhận khi cả 3 mẫu giống nhau, tức là tín hiệu đã ổn định trong ít nhất 20ms. Các xung dội ngắn hơn khoảng này bị loại bỏ.

- `keyReg3`: trạng thái ổn định đã chốt lần trước. Khi trạng thái ổn định mới khác `keyReg3`, ta có một *cạnh*. Cạnh xuống (thả → nhấn) bật cờ `button_flag`, nên mỗi lần nhấn chỉ sinh đúng một sự kiện, dù người dùng giữ nút bao lâu.

Khi nút vẫn được giữ mà không có cạnh mới, bộ đếm `hold_counter` giảm dần. Sau mỗi 1s giữ nút, cờ `long_press_flag` được bật một lần. Hàm `isButtonPressed` đọc cờ rồi xóa ngay, nên phía FSM nhận mỗi sự kiện đúng một lần.

```
#define LONG_PRESS_MS 1000
#define LONG_PRESS_TICK (LONG_PRESS_MS / TIMER_CYCLE)

static const uint16_t buttons_pin[NUM_OF_BUTTONS] = {BTN1_Pin, BTN2_Pin, BTN3_Pin};

static volatile int button_flag[NUM_OF_BUTTONS];        // ISR ghi 1, main doc roi xoa
static volatile int long_press_flag[NUM_OF_BUTTONS];
static int hold_counter[NUM_OF_BUTTONS];                // chi ISR dung

static int keyReg0[NUM_OF_BUTTONS] = {NORMAL_STATE, NORMAL_STATE, NORMAL_STATE};
static int keyReg1[NUM_OF_BUTTONS] = {NORMAL_STATE, NORMAL_STATE, NORMAL_STATE};
static int keyReg2[NUM_OF_BUTTONS] = {NORMAL_STATE, NORMAL_STATE, NORMAL_STATE};
static int keyReg3[NUM_OF_BUTTONS] = {NORMAL_STATE, NORMAL_STATE, NORMAL_STATE};

void getKeyInput(void){
    for (int i = 0; i < NUM_OF_BUTTONS; i++){
        // Dich thanh ghi: keyReg2 la mau moi nhat
        keyReg0[i] = keyReg1[i];
        keyReg1[i] = keyReg2[i];
        keyReg2[i] = HAL_GPIO_ReadPin(GPIOB, buttons_pin[i]);

        // Chi xu ly khi 3 mau giong nhau (da het doi)
        if ((keyReg0[i] == keyReg1[i]) && (keyReg1[i] == keyReg2[i])){

            if (keyReg2[i] != keyReg3[i]){
                // Co canh: trang thai on dinh moi khac lan chot truoc
                keyReg3[i] = keyReg2[i];
                if (keyReg2[i] == PRESSED_STATE){       // canh xuong = vua nhan
                    button_flag[i] = 1;
                    hold_counter[i] = LONG_PRESS_TICK;
                }
            }
            else if (keyReg2[i] == PRESSED_STATE){
                // Khong co canh va dang nhan -> dang giu nut
                hold_counter[i]--;
                if (hold_counter[i] <= 0){
                    long_press_flag[i] = 1;
                    hold_counter[i] = LONG_PRESS_TICK;  // nap lai -> bao lai moi 1s neu tiep tuc
 giu
                }
            }
        }
    }
}

int isButtonPressed(int index){
    if (index < 0 || index >= NUM_OF_BUTTONS) return 0;
    if (button_flag[index] == 1){
        button_flag[index] = 0;     // doc xong xoa -> moi lan nhan chi bao 1 lan
        return 1;
    }
    return 0;
}
```

button.c

Hàm `isButtonLongPressed` có cấu trúc giống `isButtonPressed`; điểm khác duy nhất là nó đọc và xóa cờ `long_press_flag`. Hằng `LONG_PRESS_TICK` được tính từ `TIMER_CYCLE`, nên ngưỡng nhấn giữ vẫn đúng 1s khi đổi chu kỳ ngắt (bài tập 4).

Report 3: Code tăng mode khi nhấn nút thứ nhất. Ở Mode 1, nhấn BTN1 sẽ tắt nhịp đếm giây và gọi `fsm_manual_start()` để vào Mode 2:

```
            // Nut MODE -> Mode 2
            if (isButtonPressed(BTN_MODE)){
                setTimer(TIMER_SECOND, 0);      // tat nhip dem giay cua Mode 1
                fsm_manual_start();
                break;
            }
```

Xử lý BTN1 ở Mode 1 (fsm_automatic.c)

Ở Mode 2–4, mỗi lần nhấn BTN1 chuyển sang trạng thái `MAN_*` kế tiếp. Vì 3 trạng thái `MAN_RED`, `MAN_AMBER`, `MAN_GREEN` được khai báo liên tiếp trong enum, chỉ cần `status + 1`. Từ Mode 4, hệ thống quay về Mode 1:

```
    // 2. Nut MODE: sang mode ke tiep, Mode 4 -> Mode 1
    if (isButtonPressed(BTN_MODE)){
        if (status == MAN_GREEN){
            exit_to_auto();
        }
        else{
            enter_state(status + 1);    // MAN_RED -> MAN_AMBER -> MAN_GREEN
        }
        return;
    }
```

Xử lý BTN1 ở Mode 2–4 (fsm_manual.c)

Khi vào Mode 2, hàm `flush_buttons()` xóa các cờ nút còn sót lại. Nếu không có bước này, một lần nhấn BTN2 ở Mode 1 (khi BTN2 không có tác dụng) sẽ được "để dành" và làm giá trị tăng ngay khi vừa vào Mode 2.

7 Bài tập 6

Thêm code hiển thị mode trên LED 7 đoạn, và code nhấp nháy đèn tùy theo mode được chọn.

Report 1: Hiển thị trên LED 7 đoạn. Module `led7seg` cung cấp hàm `led7seg_setPair(pair, value)` để ghi một số 0–99 lên một cặp LED: cặp 0 là LED 0–1, cặp 1 là LED 2–3. Hàm tách số thành chữ số hàng chục và hàng đơn vị rồi ghi vào `led_buffer`. Hàm `led7seg_scan()` được gọi định kỳ, mỗi lần bật một LED và tự chuyển sang LED kế tiếp.

```
void update7SEG(int index){
    if (index < 0 || index >= NUM_OF_LED7SEG) return;

```

```
    // Tat tat ca led trong luc doi doan khong LED nao sang
    for (int i = 0; i < NUM_OF_LED7SEG; i++){
        HAL_GPIO_WritePin(GPIOB, en_pins[i], GPIO_PIN_SET);
    }
    // Ghi doan cho so cua LED index
    display7SEG(led_buffer[index]);
    // Bat dung LED index -> chi LED nay hien so vua ghi
    HAL_GPIO_WritePin(GPIOB, en_pins[index], GPIO_PIN_RESET);
}

void led7seg_scan(void){
    // moi lan goi chi xu ly 1 LED, lan sau tu chuyen sang LED ke tiep
    static int scan_index = 0;

    update7SEG(scan_index);

    // Quay vong 0 -> 1 -> 2 -> 3 -> 0
    scan_index++;
    if (scan_index >= NUM_OF_LED7SEG){
        scan_index = 0;
    }
}

void led7seg_setPair(int pair, int value){
    // pair 0 -> LED 0,1 ; pair 1 -> LED 2,3 => LED dau tien cua cap = pair * 2
    if (pair < 0 || pair >= NUM_OF_LED7SEG / 2) return;

    // Gioi han 0-99 vi moi cap chi co 2 chu so
    if (value < 0)  value = 0;
    if (value > 99) value = 99;

    int first = pair * 2;
    led_buffer[first]     = value / 10; // hang chuc: 25 / 10 = 2
    led_buffer[first + 1] = value % 10; // hang don vi: 25 % 10 = 5
}
```

led7seg.c

Hàm `display7SEG` dùng bảng mã 10 chữ số, mỗi bit ứng với một đoạn (bit 0 là đoạn a, bit 6 là đoạn g). Số nằm ngoài 0–9 sẽ tắt toàn bộ đoạn. Việc quét được gọi trong `while(1)` bằng software timer `TIMER_SCAN` với `SCAN_PERIOD` = 10ms mỗi LED, nên tần số quét toàn bộ là:

$$f_{\text{quét}} = \frac{1}{4 \times 10\,\text{ms}} = 25\,\text{Hz}$$

Đây là mức Proteus mô phỏng kịp theo thời gian thực. Trên mạch thật, mắt người thấy nhấp nháy ở dưới khoảng 50Hz, nên nên giảm `SCAN_PERIOD` xuống 5ms hoặc thấp hơn.

Ở Mode 2–4, cặp LED 0–1 hiển thị giá trị đang chỉnh, cặp LED 2–3 hiển thị số mode (02, 03, 04). Số mode được tính trực tiếp từ trạng thái, vì 3 trạng thái `MAN_*` liên tiếp nhau:

```
// MAN_RED -> 2, MAN_AMBER -> 3, MAN_GREEN -> 4
static int state_mode(Traffic_State s){
    return s - MAN_RED + 2;
}

static void show_value(void){
    led7seg_setPair(0, temp_value);
    led7seg_setPair(1, state_mode(status));
```

```
}
```

Hiển thị giá trị và mode (fsm_manual.c)

Report 2: Nhấp nháy đèn theo mode. Nhấp nháy 2Hz nghĩa là chu kỳ 500ms, gồm 250ms sáng và 250ms tắt. Module `traffic_light` cung cấp hàm `traffic_toggle_color(color)` đảo trạng thái một màu ở cả hai tuyến và tắt hai màu còn lại:

```
// Pha nhay dung chung cho ca 2 huong: 1 = dang sang, 0 = dang tat
static int blink_on = 0;

void traffic_blink_reset(void){
    blink_on = 0;                   // lan toggle ke tiep chac chan BAT den
}

void traffic_toggle_color(Color color){
    if (color >= NUM_OF_COLORS) return;

    // Dung 1 bien chung thay vi HAL_GPIO_TogglePin tung chan:
    // neu truoc do 2 huong dang khac mau (vd Mode 1: do / xanh)
    // thi toggle rieng tung chan se lam 2 huong nhay lech pha nhau.
    blink_on = !blink_on;

    for (Road r = 0; r < NUM_OF_ROADS; r++){
        show_only(r, color, blink_on);
    }
}
```

Nhấp nháy đèn (traffic_light.c)

Hàm không dùng `HAL_GPIO_TogglePin` cho từng chân. Lý do: ngay trước khi vào Mode 2, hai tuyến đang khác màu (ví dụ tuyến 1 đỏ, tuyến 2 xanh). Nếu đảo riêng từng chân đỏ, đèn đỏ tuyến 1 sẽ tắt trong khi đèn đỏ tuyến 2 bật, và hai tuyến nhấp nháy lệch pha nhau. Dùng một biến `blink_on` chung bảo đảm tất cả đèn cùng màu sáng và tắt đồng thời.

Mỗi khi vào một trạng thái `MAN_*`, hàm `enter_state()` (entry action) đặt lại pha nhấp nháy, bật đèn ngay lập tức và khởi động hai software timer:

```
// Entry action cua moi trang thai MAN_*
static void enter_state(Traffic_State next){
    status = next;
    temp_value = duration[state_color(next)];

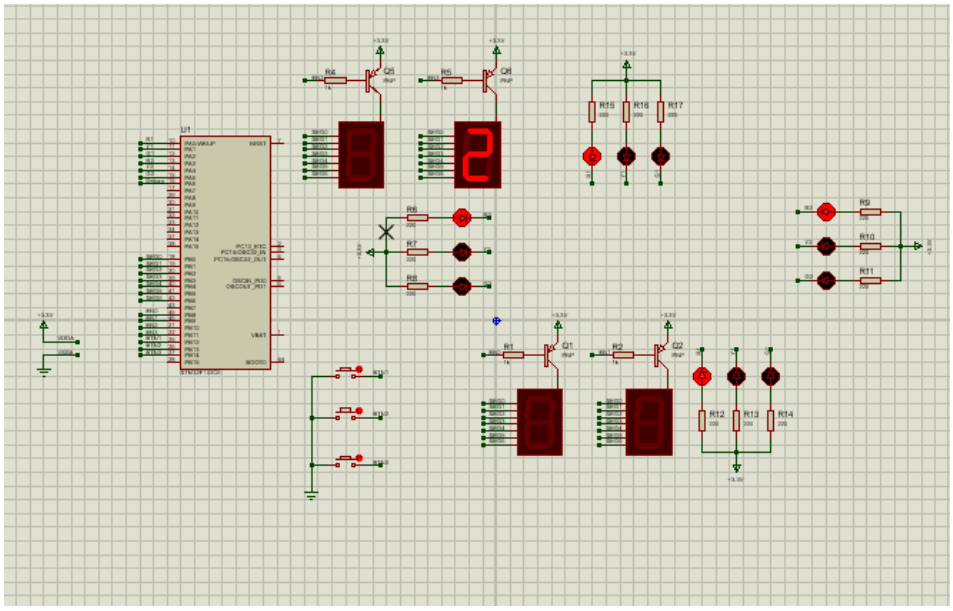
    traffic_blink_reset();
    traffic_toggle_color(state_color(next));    // sau reset -> bat den ngay
    setTimer(TIMER_BLINK, BLINK_MS);
    setTimer(TIMER_TIMEOUT, TIMEOUT_MS);

    show_value();
}
```

Entry action của các trạng thái Mode 2–4 (fsm_manual.c)

Sau đó, mỗi khi `TIMER_BLINK` hết hạn (250ms), FSM gọi lại `traffic_toggle_color` và nạp lại timer.

Kết quả mô phỏng ở Mode 2 (đèn đỏ nhấp nháy, cặp LED 2–3 hiển thị 02):



Hình 3: Hiển thị mode và nhấp nháy đèn đỏ ở Mode 2

8 Bài tập 7

Dùng nút thứ hai để tăng thời gian của đèn đỏ, và nút thứ ba để lưu giá trị.

Report 1: Khi vào trạng thái `MAN_RED`, giá trị đang chỉnh `temp_value` được khởi tạo bằng thời gian đỏ hiện tại. Giá trị này chỉ được ghi vào mảng `duration[]` khi nhấn BTN3; nếu người dùng chuyển mode mà chưa nhấn BTN3 thì thay đổi bị bỏ.

- **BTN2**: tăng `temp_value` thêm 1. Giá trị nằm trong khoảng 1–99; vượt quá 99 thì quay về 1. Giữ BTN2 thì giá trị tự tăng mỗi giây (cờ `isButtonLongPressed`).
- **BTN3**: lưu `temp_value` vào `duration[RED]`.

Mỗi lần bấm BTN2 hoặc BTN3, timer `TIMER_TIMEOUT` được nạp lại 10s, nên hệ thống chỉ tự về Mode 1 khi người dùng thật sự ngừng thao tác.

```
#define BLINK_MS    250     // 250ms sang + 250ms tat = 2Hz
#define TIMEOUT_MS  10000
#define MAX_VALUE   99
#define MIN_VALUE   1

// Gia tri dang chinh, chi ghi vao duration[] khi nhan SET
static int temp_value = 0;

static void increase_value(void){
    temp_value++;
    if (temp_value > MAX_VALUE){
        temp_value = MIN_VALUE;
    }
```

```
    show_value();
}

void fsm_manual_run(void){
    if (status != MAN_RED && status != MAN_AMBER && status != MAN_GREEN) return;

    // 1. Het 10s khong bam nut -> ve Mode 1
    if (isTimerExpired(TIMER_TIMEOUT)){
        exit_to_auto();
        return;
    }

    // 2. Nut MODE: sang mode ke tiep, Mode 4 -> Mode 1
    // ... (trinh bay o bai tap 5)

    // 3. Nut INC (nhan hoac giu): tang gia tri, dem lai timeout
    if (isButtonPressed(BTN_INC) || isButtonLongPressed(BTN_INC)){
        increase_value();
        setTimer(TIMER_TIMEOUT, TIMEOUT_MS);
    }

    // 4. Nut SET: luu gia tri, dem lai timeout
    if (isButtonPressed(BTN_SET)){
        duration[state_color(status)] = temp_value;
        setTimer(TIMER_TIMEOUT, TIMEOUT_MS);
    }

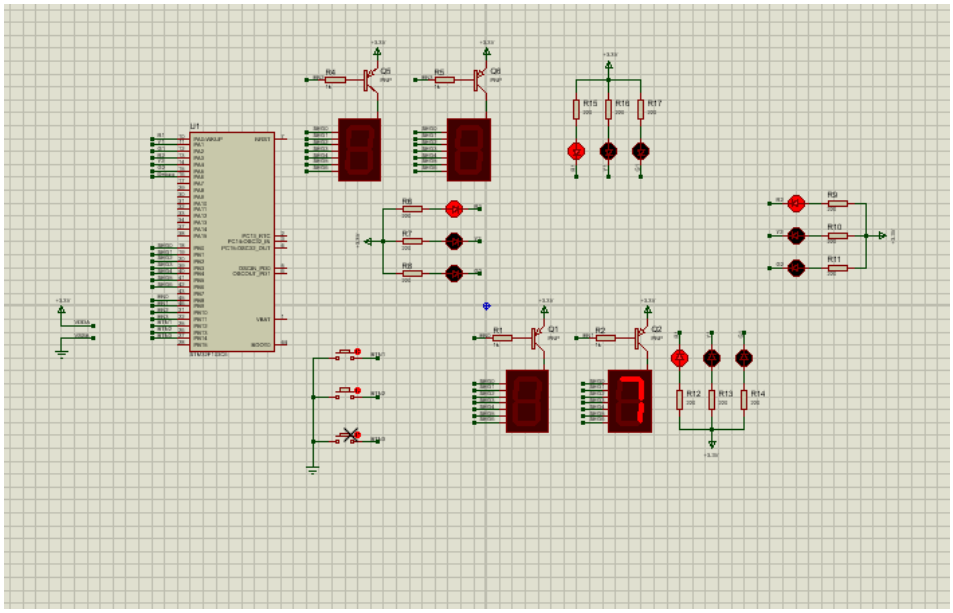
    // 5. Nhay den dang chinh
    if (isTimerExpired(TIMER_BLINK)){
        setTimer(TIMER_BLINK, BLINK_MS);
        traffic_toggle_color(state_color(status));
    }
}
```

Tăng và lưu giá trị (fsm_manual.c)

Hàm `state_color()` đổi trạng thái thành chỉ số màu, ví dụ `MAN_RED` → `RED`. Nhờ vậy, lệnh lưu giá trị dưới đây dùng chung cho cả 3 mode, không cần viết riêng cho từng màu:

```
duration[state_color(status)] = temp_value;
```

Kết quả mô phỏng: chỉnh thời gian đèn đỏ ở Mode 2.



Hình 4: Chỉnh thời gian đèn đỏ ở Mode 2

9 Bài tập 8

Dùng nút thứ hai để tăng thời gian của đèn vàng, và nút thứ ba để lưu giá trị.

Report 1: Mode 3 dùng chung toàn bộ code của Mode 2 ở bài tập 7, chỉ khác trạng thái là `MAN_AMBER`. Vì các trạng thái `MAN_*` và các màu được khai báo cùng thứ tự trong enum, một phép trừ là đủ để đổi trạng thái thành màu:

```
// MAN_RED -> RED, MAN_AMBER -> AMBER, MAN_GREEN -> GREEN
static Color state_color(Traffic_State s){
    return (Color)(s - MAN_RED);
}
```

Ánh xạ trạng thái sang màu (fsm_manual.c)

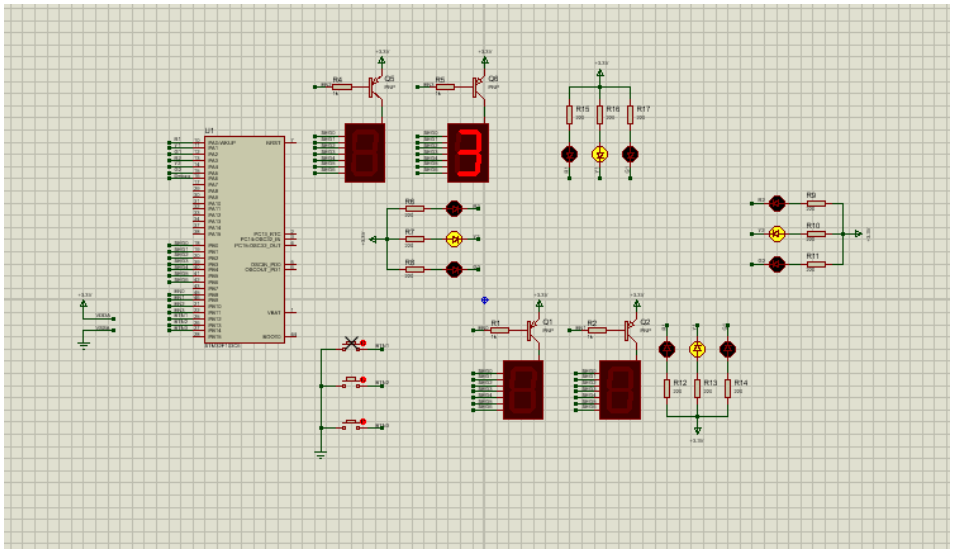
```
// Trang thai he thong: dung chung cho fsm_automatic va fsm_manual
typedef enum{
    INIT = 0,
    AUTO_RED_GREEN = 1, AUTO_RED_AMBER, AUTO_GREEN_RED, AUTO_AMBER_RED, // Mode 1
    MAN_RED = 11, MAN_AMBER, MAN_GREEN                                  // Mode 2, 3, 4
} Traffic_State;

// Chi so mau, dung cho duration[] va traffic_light
typedef enum{ RED = 0, AMBER, GREEN, NUM_OF_COLORS } Color;
```

Khai báo trạng thái và màu (global.h)

Ở `MAN_AMBER`: đèn vàng ở cả hai tuyến nhấp nháy 2Hz, cặp LED 2–3 hiển thị `03`, BTN2 tăng giá trị và BTN3 lưu vào `duration[AMBER]`.

Kết quả mô phỏng: chỉnh thời gian đèn vàng ở Mode 3.



Hình 5: Chỉnh thời gian đèn vàng ở Mode 3

10 Bài tập 9

Dùng nút thứ hai để tăng thời gian của đèn xanh, và nút thứ ba để lưu giá trị.

Report 1: Tương tự bài tập 8, Mode 4 ứng với trạng thái `MAN_GREEN`: đèn xanh nhấp nháy, cặp LED 2–3 hiển thị 04, BTN3 lưu giá trị vào `duration[GREEN]`. Điểm khác biệt là Mode 4 là mode chỉnh cuối cùng: nhấn BTN1 ở đây sẽ thoát về Mode 1 thông qua hàm `exit_to_auto()`.

Trước khi về Mode 1, hàm này kiểm tra ràng buộc $T_{\text{đỏ}} = T_{\text{xanh}} + T_{\text{vàng}}$. Nếu bộ thời gian mới không thỏa, toàn bộ thay đổi bị hủy và mảng `duration[]` được khôi phục từ bản sao `backup[]` đã lưu lúc vào Mode 2:

```
// Ban sao duration[] luc vao Mode 2, de khoi phuc neu cac gia tri moi sai rang buoc
static int backup[NUM_OF_COLORS];

// Roi Mode 2-4, ve Mode 1: tat timer rieng, fsm_automatic lo phan con lai tai INIT
static void exit_to_auto(void){
    // Rang buoc: den do 1 huong = xanh + vang huong kia, sai thi khoi phuc gia tri cu
    if (duration[RED] != duration[AMBER] + duration[GREEN]){
        for (int c = 0; c < NUM_OF_COLORS; c++){
            duration[c] = backup[c];
        }
    }
    setTimer(TIMER_BLINK, 0);
    setTimer(TIMER_TIMEOUT, 0);
    traffic_all_off();
    status = INIT;
}

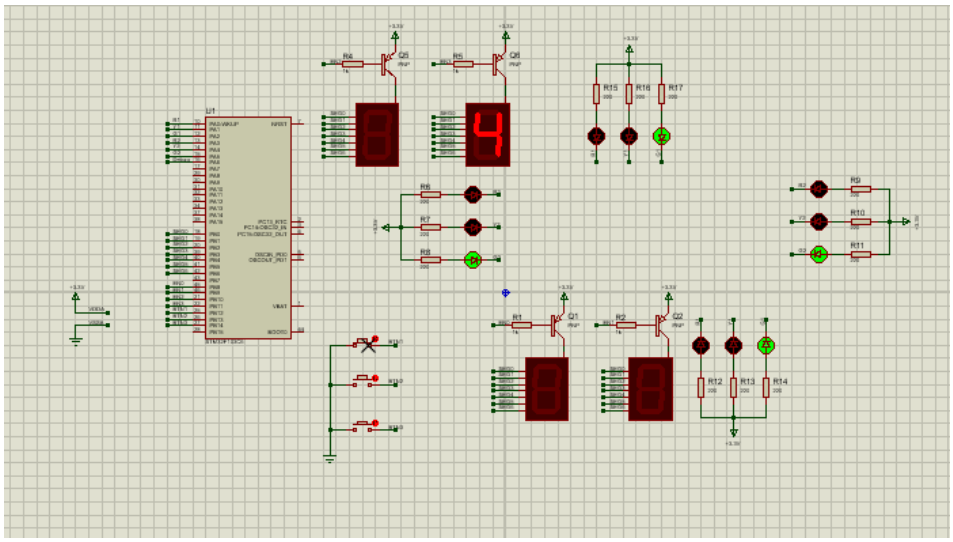
void fsm_manual_start(void){
```

```
    for (int c = 0; c < NUM_OF_COLORS; c++){
        backup[c] = duration[c];
    }
    flush_buttons();
    enter_state(MAN_RED);
}
```

Thoát về Mode 1 (fsm_manual.c)

Ví dụ, từ bộ mặc định (đỏ 5, vàng 2, xanh 3), muốn có đỏ 7s và xanh 5s: ở Mode 2 chỉnh và lưu đỏ $= 7$, Mode 3 giữ vàng $= 2$, Mode 4 chỉnh và lưu xanh $= 5$. Khi về Mode 1, $7 = 5 + 2$ nên bộ thời gian mới được áp dụng. Nếu người dùng chỉ đổi đỏ thành 7 mà quên chỉnh xanh, ràng buộc bị vi phạm ($7 \neq 3 + 2$) và hệ thống dùng lại bộ cũ (5, 2, 3), thay vì chạy với hai tuyến lệch pha.

Kết quả mô phỏng: chỉnh thời gian đèn xanh ở Mode 4.



Hình 6: Chỉnh thời gian đèn xanh ở Mode 4

11 Bài tập 10

Tích hợp toàn bộ các bài tập trước thành một project hoàn chỉnh, quay video trình bày các chức năng theo đặc tả.

Report 1: Vòng lặp chính chỉ gồm ba việc: quét LED 7 đoạn khi `TIMER_SCAN` hết hạn, chạy máy trạng thái Mode 1 và chạy máy trạng thái Mode 2–4. Mỗi hàm FSM tự kiểm tra `status` và chỉ xử lý khi trạng thái thuộc phần của mình, nên hai hàm có thể gọi nối tiếp mà không cần `if/else` ở hàm main.

```
/* USER CODE BEGIN PD */
// Chu ky quet 1 LED 7 doan (ms). Proteus: 10ms la vua; phan cung that nen <= 5ms
#define SCAN_PERIOD 10
/* USER CODE END PD */

```

```
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
      if (isTimerExpired(TIMER_SCAN)){
          setTimer(TIMER_SCAN, SCAN_PERIOD);
          led7seg_scan();
      }
      fsm_automatic_run();
      fsm_manual_run();
  }
  /* USER CODE END 3 */
```

Vòng lặp chính (main.c)

Report 2: Máy trạng thái Mode 1. Mỗi tuyến có một biến đếm ngược riêng (`time_road1`, `time_road2`), giảm mỗi giây theo `TIMER_SECOND` và được hiển thị lên cặp LED tương ứng. Khi vào một pha mới, hàm `enter_phase()` chỉ nạp lại thời gian cho tuyến *vừa đổi màu*; tuyến còn lại vẫn đếm tiếp. Ví dụ ở pha `AUTO_RED_AMBER`, tuyến 1 vẫn đỏ nên tiếp tục đếm từ 2 về 1, trong khi tuyến 2 được nạp thời gian vàng. Nhờ ràng buộc $T_{\text{đỏ}} = T_{\text{xanh}} + T_{\text{vàng}}$, đèn đỏ của tuyến 1 và đèn vàng của tuyến 2 về 0 cùng lúc.

```
// Entry action: chuyen sang pha moi, bat den va nap so giay cho den vua doi mau.
// Den khong doi mau thi giu nguyen so giay dang dem.
static void enter_phase(Traffic_State next){
    status = next;
    switch (next){
        case AUTO_RED_GREEN:
            time_road1 = duration[RED];
            time_road2 = duration[GREEN];
            traffic_set(ROAD_1, RED);
            traffic_set(ROAD_2, GREEN);
            break;
        case AUTO_RED_AMBER:
            time_road2 = duration[AMBER];       // huong 1 van do, dem tiep
            traffic_set(ROAD_2, AMBER);
            break;
        case AUTO_GREEN_RED:
            time_road1 = duration[GREEN];
            time_road2 = duration[RED];
            traffic_set(ROAD_1, GREEN);
            traffic_set(ROAD_2, RED);
            break;
        case AUTO_AMBER_RED:
            time_road1 = duration[AMBER];       // huong 2 van do, dem tiep
            traffic_set(ROAD_1, AMBER);
            break;
        default:
            break;
    }
    show_time();
}

// Pha hien tai ket thuc khi den vua doi mau dem ve 0
static int phase_done(void){
    switch (status){
        case AUTO_RED_GREEN:
        case AUTO_RED_AMBER:    return time_road2 <= 0; // huong 2 doi mau
```

```
        case AUTO_GREEN_RED:
        case AUTO_AMBER_RED:    return time_road1 <= 0; // huong 1 doi mau
        default:                return 0;
    }
}

void fsm_automatic_run(void){
    switch (status){
        case INIT:
            // Vao Mode 1 (khoi dong hoac tu Mode 2-4 quay ve)
            check_duration();
            setTimer(TIMER_SECOND, SECOND_MS);
            enter_phase(AUTO_RED_GREEN);
            break;

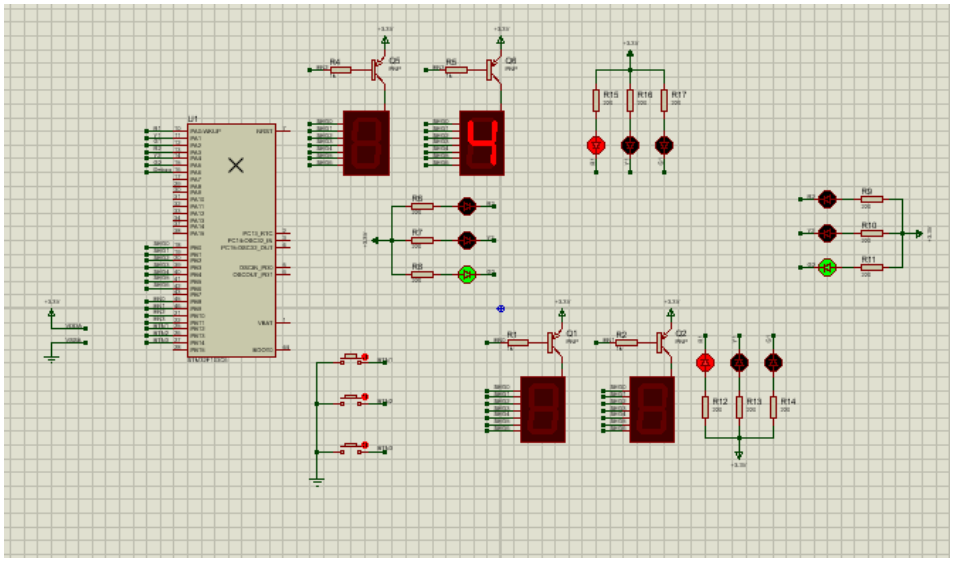
        case AUTO_RED_GREEN:
        case AUTO_RED_AMBER:
        case AUTO_GREEN_RED:
        case AUTO_AMBER_RED:
            // Nut MODE -> Mode 2
            if (isButtonPressed(BTN_MODE)){
                setTimer(TIMER_SECOND, 0);      // tat nhip dem giay cua Mode 1
                fsm_manual_start();
                break;
            }
            // Moi 1s: dem nguoc ca 2 huong, het pha thi chuyen pha
            if (isTimerExpired(TIMER_SECOND)){
                setTimer(TIMER_SECOND, SECOND_MS);
                time_road1--;
                time_road2--;
                if (phase_done()){
                    enter_phase(next_phase(status));
                }
                else{
                    show_time();
                }
            }
            break;

        default:
            break;      // MAN_*: do fsm_manual xu ly
    }
}
```

fsm_automatic.c

Hàm `next_phase()` trả về pha kế tiếp theo vòng `RED_GREEN` → `RED_AMBER` → `GREEN_RED` → `AMBER_RED`. Hàm `check_duration()` là lớp bảo vệ cuối: nếu `duration[]` vi phạm ràng buộc vì bất kỳ lý do gì, Mode 1 sẽ chạy với bộ mặc định (5, 2, 3).

Kết quả mô phỏng ở Mode 1 (pha `AUTO_RED_GREEN`: tuyến 1 đỏ, tuyến 2 xanh, LED 7 đoạn đếm ngược):



Hình 7: Hoạt động của hệ thống ở Mode 1

Video trình bày các chức năng của hệ thống: https://youtu.be/rKUgLeOr6Bk

12 Mã nguồn cho bài thực hành 3

Toàn bộ mã nguồn (STM32CubeIDE project) cho bài thực hành 3: https://github.com/qanetta/Microprocessor-Lab/tree/main/lab3/stm32cube_src

Thư mục gồm hai project có cùng chức năng và dùng chung một sơ đồ Proteus:

- `Traffic-Light_FSM`: project được trình bày trong báo cáo này, tách module theo vai trò (`button`, `software_timer`, `traffic_light`, `led7seg`, `fsm_automatic`, `fsm_manual`).
- `Traffic-Light_Lab-default`: hiện thực theo cấu trúc file gợi ý trong tài liệu thực hành (`input_reading`, `input_processing`, `led_display`, `timer`).

File mô phỏng Proteus cho bài thực hành 3: https://github.com/qanetta/Microprocessor-Lab/tree/main/lab3/proteus_files